Aaron N. Cutshall, DHA, MSHI, ACHE

SQL RV http://www.sqlrv.com

Migrating AdventureWorks to Postgres – Part 1



SQLRV, LLC

HEALTHCARE INFORMATICS

https://www.sqlrv.com



Just who is this guy?

SQLRV, LLC
HEALTHCARE INFORMATICS

Principal Data Architect, Population Health

BYU PATHWAY WORLDWIDE

Adjunct Instructor

redgate AMBASSADOR

Ambassador Program

DGIQ + EDW DATA SUMMIT
UNLEASH THE POWER OF YOUR DATA

Speaker – various events

B.S. Computer Science

M.S. Computer Information Systems

M.S. Health Informatics

VIRGINIA UNIVERSITY OF LYNCHBURG · SIBI AUXILIUM · ET LIBERTAS · 1886

Doctor of Healthcare Administration

40 Years



Copyright © 2026, SQLRV, LLC - All Rights Reserved

Something to consider…



credit

“You must go on adventures to find out where you truly belong.”

– Sue Fitzmaurice (author, editor and publisher)

Copyright © 2024, SQL RV - All Rights Reserved

Agenda

- Goals of this session
  - Introduce the AdventureWorks Migration project
  - Article series for SimpleTalk
  - Showcase the strengths/weaknesses of both SQL Server and Postgres
- AdventureWorks
  - What is it?
  - Why move it to Postgres?
  - What are the challenges?



Copyright © 2024, SQL RV - All Rights Reserved

Agenda

- Migration Milestones
  - Schemas
  - Tables
  - Data Types
  - Custom Data Types
  - Data Load
  - Primary Keys
  - Indexes
  - Foreign Keys



Copyright © 2024, SQL RV - All Rights Reserved

AdventureWorks

- What is it?
  - Serves as a demonstration of many SQL Server features
  - Provides a testing platform to debug application issues
- Why move it to Postgres?
  - To demonstrate the differences between the two platforms
  - Provide a similar testing platform for Postgres
- What are the challenges?
  - Not all features align between the two platforms
  - Some features are implemented differently or not at all



Copyright © 2024, SQL RV - All Rights Reserved

Migration Milestones

- Schemas
  - More pronounced usage in Postgres than SQL Server
  - Provides more organization capabilities
  - Can be somewhat treated like mini-databases
- Tables
  - Basically similar between Postgres and SQL Server
  - Quote mechanism is different – double quotes vs square brackets

Copyright © 2024, SQL RV - All Rights Reserved



Migration Milestones

- Data Types
  - Many are identical (date, int, decimal, etc.)
  - Some have different names with some minor differences
    - NCHAR / NVARCHAR vs char / varchar
    - DATETIME vs timestamp
    - BIT vs boolean
    - GUID vs uuid
  - Some seem to be the same but are implemented differently
    - GEOGRAPHY vs geography
    - HIERARCHYID vs ltree

Copyright © 2024, SQL RV - All Rights Reserved



Migration Milestones

- Data Types (cont.)
  - Some types do not exist so substitutions must be made

    Ex: `MONEY` → `numeric(20,4)` (money not recommended)
  - Identity columns are treated a little differently

    `IDENTITY(1,1)` → `GENERATED BY DEFAULT AS IDENTITY`
  - Defaults in Postgres are not constraints
  - Postgres has one type (bytea) for all binary data
  - Postgres supports XML but not schema (XSD) validation



Copyright © 2024, SQL RV - All Rights Reserved

Migration Milestones

- Custom Data Types
  - SQL Server

    ```
    CREATE TYPE TypeName FROM DataType NullConstraint;
    ```

    Example:

    ```
    CREATE TYPE [Flag] FROM BIT NOT NULL;
    ```

  - Postgres

    ```
    CREATE DOMAIN DomainName FROM DataType NullConstraint;
    ```

    Example:

    ```
    CREATE DOMAIN flag FROM boolean NOT NULL;
    ```



Copyright © 2024, SQL RV - All Rights Reserved

Migration Milestones

- Example Tables:

```
-- SQL Server
CREATE TABLE [Person].[Address](
    [AddressID] [int] IDENTITY (1, 1) NOT FOR REPLICATION NOT NULL,
    [AddressLine1] [nvarchar](60) NOT NULL,
    [AddressLine2] [nvarchar](60) NULL,
    [City] [nvarchar](30) NOT NULL,
    [StateProvinceID] [int] NOT NULL,
    [PostalCode] [nvarchar](15) NOT NULL,
    [SpatialLocation] [geography] NULL,
    [rowguid] uniqueidentifier ROWGUIDCOL NOT NULL CONSTRAINT [DF_Address_rowguid] DEFAULT (NEWID()),
    [ModifiedDate] [datetime] NOT NULL CONSTRAINT [DF_Address_ModifiedDate] DEFAULT (GETDATE())
) ON [PRIMARY];

-- Postgres
CREATE TABLE "Person"."Address" (
    "AddressID" INT GENERATED BY DEFAULT AS IDENTITY,
    "AddressLine1" varchar(60) NOT NULL,
    "AddressLine2" varchar(60) NULL,
    "City" varchar(30) NOT NULL,
    "StateProvinceID" int NOT NULL,
    "PostalCode" varchar(15) NOT NULL,
    "SpatialLocation" geography NULL, -- Representing longitude and latitude
    rowguid uuid not null DEFAULT gen_random_uuid(), -- constraint name ignored as defaults are not constraints in Postgres
    "ModifiedDate" timestamp NOT NULL DEFAULT now()
  );
```

Copyright © 2024, SQL RV - All Rights Reserved



Migration Milestones

- Example Tables (cont.):

```
-- SQL Server
CREATE TABLE [Production].[Document](
    [DocumentNode] [hierarchyid] NOT NULL,
    [DocumentLevel] AS DocumentNode.GetLevel(),
    [Title] [nvarchar](50) NOT NULL,
    [Owner] [int] NOT NULL,
    [FolderFlag] [bit] NOT NULL CONSTRAINT [DF_Document_FolderFlag] DEFAULT (0),
    [FileName] [nvarchar](400) NOT NULL,
    [FileExtension] nvarchar(8) NOT NULL,
    [Revision] [nchar](5) NOT NULL,
    [ChangeNumber] [int] NOT NULL CONSTRAINT [DF_Document_ChangeNumber] DEFAULT (0),
    [Status] [tinyint] NOT NULL,
    [DocumentSummary] [nvarchar](max) NULL,
    [Document] [varbinary](max)  NULL,
    [rowguid] uniqueidentifier ROWGUIDCOL NOT NULL UNIQUE CONSTRAINT [DF_Document_rowguid] DEFAULT (NEWID()),
    [ModifiedDate] [datetime] NOT NULL CONSTRAINT [DF_Document_ModifiedDate] DEFAULT (GETDATE()),
    CONSTRAINT [CK_Document_Status] CHECK ([Status] BETWEEN 1 AND 3)
) ON [PRIMARY];
```



Copyright © 2024, SQL RV - All Rights Reserved

Migration Milestones

- Example Tables (cont.):

```
-- Postgres
CREATE TABLE "Production"."Document" (
    "DocumentNode" ltree NOT NULL,
    "DocumentLevel" int NOT NULL GENERATED ALWAYS AS (nlevel("DocumentNode")) STORED,
    "Title" varchar(50) NOT NULL,
    "Owner" int NOT NULL,
    "FolderFlag" boolean NOT NULL DEFAULT false,
    "FileName" varchar(400) NOT NULL,
    "FileExtension" varchar(8) NOT NULL,
    "Revision" char(5) NOT NULL,
    "ChangeNumber" int NOT NULL DEFAULT 0,
    "Status" smallint NOT NULL CONSTRAINT "CK_Document_Status" CHECK ("Status" BETWEEN 1 AND 3),
    "DocumentSummary" text NULL,
    "Document" bytea NULL,
    rowguid uuid NOT NULL DEFAULT gen_random_uuid(),
    "ModifiedDate" timestamp NOT NULL DEFAULT now()
    );
```

Copyright © 2024, SQL RV - All Rights Reserved



Migration Milestones

- Populate Tables
  - SQL Server uses BULK INSERT with many options
    - Multiple configurations used for SQL Server
    - Intended to highlight multiple methods
  - Postgres uses COPY command
    - All data, except a few, are converted to CSV format
    - Pipe delimiter used in some cases where commas exist in text
  - Some data conversions were necessary
    - GEOGRAPHY → Well-Known Text → geography
    - GUID → text → uuid



Copyright © 2024, SQL RV - All Rights Reserved

Migration Milestones

- Primary Keys
  - Usually defined with column:

    ```
    column_1 data_type PRIMARY KEY
    ```

  - Can be part of table definition:

    ```
    PRIMARY KEY(column_1, column_2, …)
    ```

  - Added after table creation

    ```
    ALTER TABLE table_name
       ADD PRIMARY KEY(column_1, column_2, …);
    ```

    - Used when data is populated first
    - Index is created when the primary key is defined



Copyright © 2024, SQL RV - All Rights Reserved

Migration Milestones

- Indexes
  - Clustered indexes not defined in Postgres
  - Default index type is B-Tree, just like SQL Server
  - Other index types supported:
    - HASH – 32-bit value for only equality comparisons
    - GiST – Supports multiple indexing strategies and methods
    - GIN – Inverted indexes for multiple custom methods
    - BRIN – Block Range Indexes can be made similar to clustering
  - Syntax very much like SQL Server



Copyright © 2024, SQL RV - All Rights Reserved

Migration Milestones

- Foreign Keys
  - Essentially a reference to another table's Primary Key
  - Implemented very similar in Postgres as in SQL Server

```
-- SQL Server
ALTER TABLE [Production].[BillOfMaterials] ADD
    CONSTRAINT [FK_BillOfMaterials_Product_ProductAssemblyID]
      FOREIGN KEY ([ProductAssemblyID]) REFERENCES [Production].[Product]([ProductID]),
    CONSTRAINT [FK_BillOfMaterials_Product_ComponentID]
      FOREIGN KEY ([ComponentID]) REFERENCES [Production].[Product]([ProductID]),
    CONSTRAINT [FK_BillOfMaterials_UnitMeasure_UnitMeasureCode]
      FOREIGN KEY ([UnitMeasureCode]) REFERENCES [Production].[UnitMeasure]([UnitMeasureCode]);
-- Postgres
ALTER TABLE "Production"."BillOfMaterials" ADD
    CONSTRAINT "FK_BillOfMaterials_Product_ProductAssemblyID"
      FOREIGN KEY ("ProductAssemblyID") REFERENCES "Production"."Product"("ProductID"),
    CONSTRAINT "FK_BillOfMaterials_Product_ComponentID"
      FOREIGN KEY ("ComponentID") REFERENCES "Production"."Product"("ProductID"),
    CONSTRAINT "FK_BillOfMaterials_UnitMeasure_UnitMeasureCode"
      FOREIGN KEY ("UnitMeasureCode") REFERENCES "Production"."UnitMeasure"("UnitMeasureCode");
```



Copyright © 2024, SQL RV - All Rights Reserved

Portions Not Covered

- Migrating data is easy; the rest is hard!
- Views – Differences in queries
- Triggers – Introduction to PL/pgSQL
- Functions – Scalar, Multi-Statement and Inline Tabular
- Stored Procedures
- Miscellaneous tasks
  - Tracking database changes
  - Object Comments
  - Other housekeeping duties



Copyright © 2024, SQL RV - All Rights Reserved

Migration Challenges

- AdventureWorks was explicitly designed for SQL Server
- Keeping the purpose of AdventureWorks
  - Preserving the same names and conventions
  - Matching equivalent data types and constructs
  - Converting non-conformant data
- Migration was to introduce Postgres and how to move from SQL Server
- Certainly not for the faint of heart!



Copyright © 2024, SQL RV - All Rights Reserved

Closing Thoughts

- The initial phase was to load the database
- Introduction to query differences and PL/pgSQL in subsequent parts
- Read the series on SimpleTalk for further details
- Many books on both SQL Server and Postgres
- Not as much written about migrating from SQL Server to Postgres
- Will consider writing a book on the subject if there's strong enough interest



Copyright © 2024, SQL RV - All Rights Reserved

References

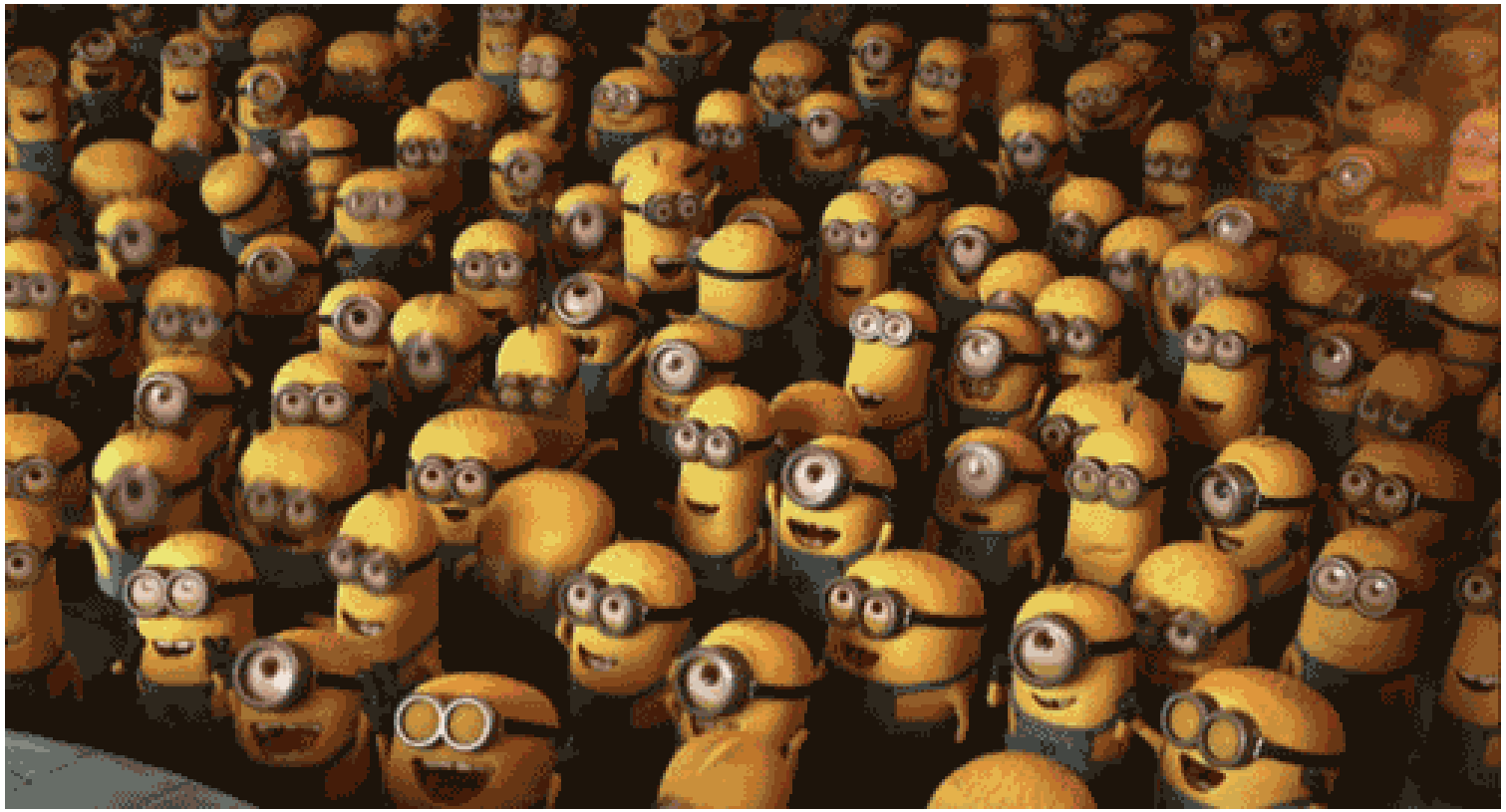
- https://blog.devart.com/postgresql-vs-sql-server.html
- https://www.mssqltips.com/sqlservertip/7886/postgresql-history-features-and-sql-server-comparison/
- https://www.sqlines.com/sql-server-to-postgresql
- https://stackoverflow.com/questions/1370326/migrate-data-from-ms-sql-to-postgresql
- https://dzone.com/articles/sql-server-to-postgres-database-migration
- https://estuary.dev/sql-server-to-postgres/
- https://dev.to/abhinavgupta1997/migrating-database-from-sql-server-mssql-to-postgresql-1mje



Copyright © 2024, SQL RV - All Rights Reserved

Thanks to our Sponsors!

Without them, we wouldn’t be here!!



Copyright © 2026, SQLRV LLC - All Rights Reserved

Questions & Comments

BONUS:
A **TON** of free eBooks from RedGate and others (databases and programming)!

PRESENTATION FEEDBACK:
- Your thoughts needed
- Improve presentations
- Make this event even more valuable!!!

Aaron N. Cutshall

www.linkedin.com/in/sqlrv
aaron@sqlrv.com
www.sqlrv.com
www.youtube.com/@sqlrv



Copyright © 2026, SQLRV LLC - All Rights Reserved